# Recursive algorithms (1/2)

# Outline

The sum of the elements of an array

Merge Sort

# From the previous chapters

In the chapters «Recursion», «Recurrences» and «The Master Theorem» we have learnt:

- how a recursive algorithm is executed
- how its number of steps is described by a **recurrence**
- how such a recurrence is solved

In this chapter we put these tools to work on four classical problems on arrays.

# Four classical problems on arrays

- the **sum** of the elements of an array, the simplest possible problem, on which we compare an iterative and two recursive algorithms
- **sorting**, with **Merge Sort**, the first algorithm of the course that beats the $\Theta(n^2)$ sorting algorithms
- **searching** in a sorted array, with the **Recursive Binary Search**
- the **maximum subarray** problem, where a divide and conquer algorithm improves on brute force, and a clever iterative algorithm improves on both

This lesson: the sum of an array and Merge Sort.

# The scheme of the analysis

All the algorithms are studied with the same scheme, which is the scheme of the analysis of every recursive algorithm. For each recursive algorithm we:

1. write the pseudo-code and run it on a small example, drawing the **call tree**;
2. prove that it is **correct**, by (strong) induction on the size of the sub-problem;
3. describe its number of steps by a **recurrence** and solve it, with the recursion tree and with the Master Theorem;
4. express the worst-case **running time** as a function of the input size $|I|$;
5. measure the **extra memory**, which for a recursive algorithm includes the call stack;
6. give an implementation (arrays indexed from 0, while in the pseudo-code they are indexed from 1): the Colab notebooks in Python, C++ and Java.

# Divide and conquer

- Three of the four algorithms follow the **divide and conquer** paradigm:
  - the problem on a sub-array $a[l..r]$ is split into sub-problems on the two halves $a[l..m]$ and $a[m+1..r]$ (*divide*)
  - the sub-problems are solved recursively (*conquer*)
  - their solutions are put together (*combine*)
- The Recursive Binary Search, instead, makes a **single** recursive call on one half

# The sum of the elements of an array

# Introduction: The sum of the elements of an array

- The simplest problem on arrays: we use it to compare an **iterative** algorithm and a **recursive** algorithm that solve the same problem
- On this problem we go through the whole scheme of the analysis: pseudo-code, call tree, correctness, recurrence, recursion tree, Master Theorem, running time
- We discover that recursion does not make the algorithm faster, and that the **call stack** costs memory

# The problem

Given an array $a[1..n]$ of $n \geq 1$ integer numbers,
we consider the problem of computing the sum of its elements:

$$\sum_{i=1}^{n} a[i] = a[1] + a[2] + \cdots + a[n].$$

- It is the ideal problem to compare an **iterative** and a **recursive** algorithm
- We write the recursive algorithm, prove that it is correct, describe its number of steps by a recurrence and solve the recurrence, with the tools of the chapters «Recursion», «Recurrences» and «The Master Theorem»

# Example

We consider the following array of $n = 7$ integer numbers:

| $a[i]$ | $i = 1$ | $i = 2$ | $i = 3$ | $i = 4$ | $i = 5$ | $i = 6$ | $i = 7$ |
|---|---|---|---|---|---|---|---|
| | 9 | 8 | 12 | 3 | 5 | 14 | 6 |

The sum of its elements is

$$9 + 8 + 12 + 3 + 5 + 14 + 6 = 57.$$

# Iterative algorithm

We start from the obvious iterative algorithm `SumIterative`,
which scans the array from left to right
and accumulates the partial sums in the variable $s$.

**Algorithm:** SumIterative

**Input** : An array $a[1..n]$ of $n$ integer numbers
**Output** : The sum $a[1] + a[2] + \cdots + a[n]$

```
1 function SumIterative(a, n)
2     s := 0
3     i := 1
4     while i ≤ n do
5         s ← s + a[i]
6         i ← i + 1
7     end
8     return s
9 end
```

Run in Colab: Python · C++ · Java

# Number of steps of `SumIterative`

- Let $S(n)$ denote the number of steps executed by `SumIterative` on an array of $n$ elements (no difference between best and worst case: the algorithm always performs the same operations)

| line | number of executions |
|---|---|
| 2 | 1 |
| 3 | 1 |
| 4 | $n+1$ |
| 5 | $n$ |
| 6 | $n$ |
| 8 | 1 |

# Steps and memory of `SumIterative`

- Therefore,

$$S(n) = 1 + 1 + (n + 1) + n + n + 1 = 3n + 4 = \Theta(n),$$

since

$$\lim_{n\to\infty} \frac{3n+4}{n} = \lim_{n\to\infty} \left(3 + \frac{4}{n}\right) = 3 \in \mathbb{R}_{>0}.$$

- Besides the input array, the algorithm uses only the two variables $s$ and $i$: the **extra memory** is $\Theta(1)$

# Recursive algorithm (divide and conquer)

- The same sum can be computed recursively. The sum of the elements of a sub-array $a[l..r]$ is obtained by **divide and conquer**:
  1. **Divide**: split $a[l..r]$ in two halves $a[l..m]$ and $a[m+1..r]$, where $m = \lfloor (l+r)/2 \rfloor$;
  2. **Conquer**: compute recursively the sum $s_1$ of $a[l..m]$ and the sum $s_2$ of $a[m+1..r]$;
  3. **Combine**: return $s_1 + s_2$.

- The **base case** is a sub-array with a single element ($l = r$): its sum is the element $a[l]$ itself

- In the next slide, the pseudo-code of the recursive algorithm `RecursiveSum`. To sum the whole array we call `RecursiveSum`$(a, 1, n)$

**Algorithm:** RecursiveSum

**Input :** An array $a$ and two indices $l \leq r$
**Output :** The sum $a[l] + a[l+1] + \cdots + a[r]$

```
1 function RecursiveSum(a, l, r)
2     if l = r then
3         return a[l]                                  // Base case
4     end
5     m := ⌊(l + r)/2⌋                                 // Divide
6     s1 := RecursiveSum(a, l, m)   // Conquer (left half)
7     s2 := RecursiveSum(a, m + 1, r)    // Conquer (right
       half)
8     return s1 + s2                                   // Combine
9 end
```

Run in Colab: Python · C++ · Java

# The recursion always reaches the base case

- Let $k = r - l + 1$ be the number of elements of $a[l..r]$
- If $k \geq 2$, the two halves have

$$m - l + 1 = \left\lceil \frac{k}{2} \right\rceil \qquad \text{and} \qquad r - m = \left\lfloor \frac{k}{2} \right\rfloor$$

  elements, respectively

- Both numbers are at least 1 and strictly smaller than $k$: every recursive call works on a smaller sub-array, and the recursion always reaches the base case

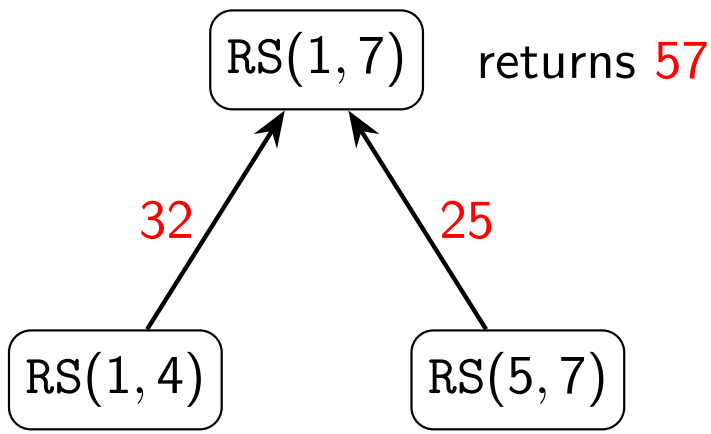
# Call tree

- The execution of a recursive algorithm is represented by its **call tree**:
  - every node is a call
  - the children of a node are the calls it makes
  - the value returned by a call is sent back to its parent

- In the next example we draw the call tree of `RecursiveSum` on the array $a = [9, 8, 12, 3, 5, 14, 6]$, with $n = 7$

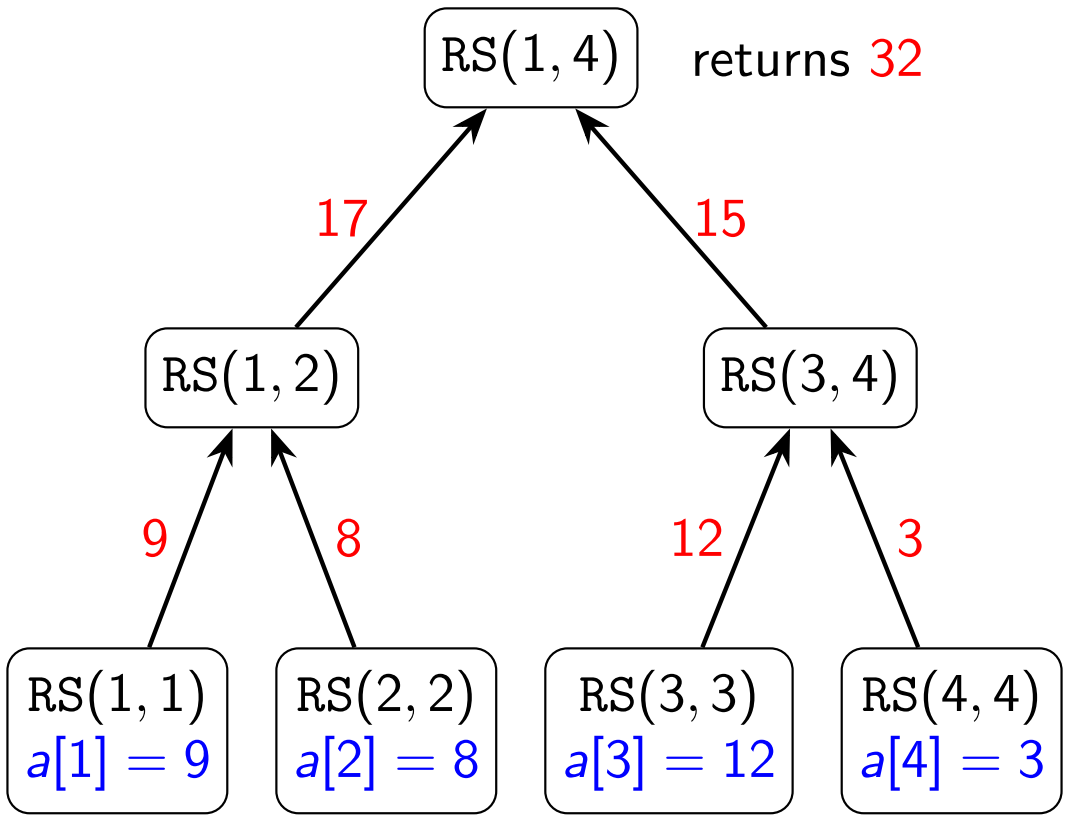
# Example (1/4)

- The call tree of `RecursiveSum`$(a, 1, 7)$: each node RS$(l, r)$ is a call on the sub-array $a[l..r]$
- In blue the elements of the array returned by the base cases, in red the values returned by the calls to their parents (along the arrows)
- The tree is too large for one slide: we draw first its first two levels, then the sub-tree of RS$(1, 4)$ and the sub-tree of RS$(5, 7)$



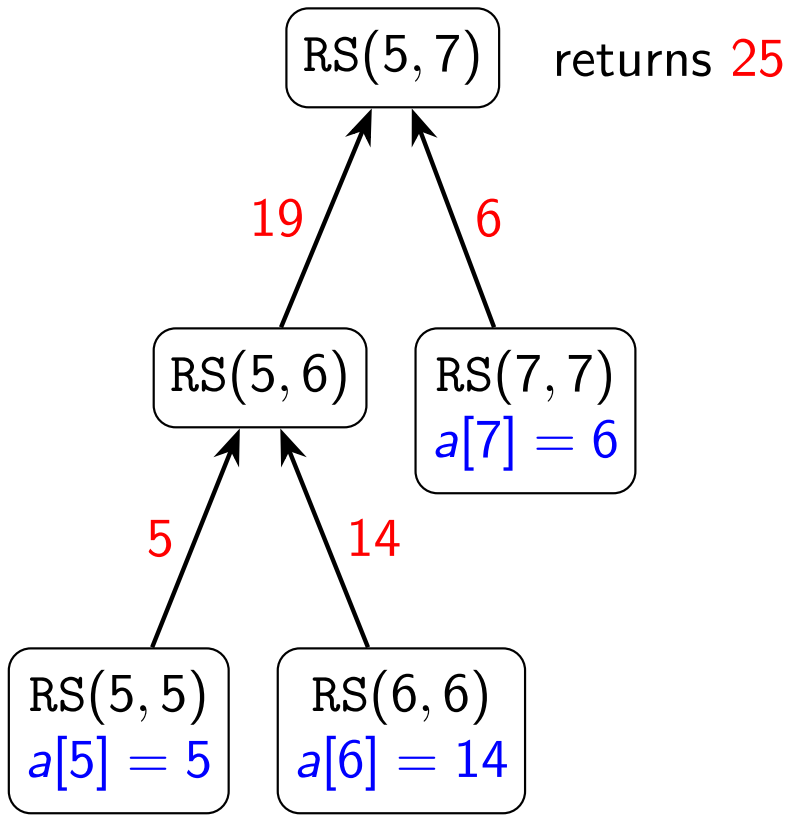
# Example (2/4)

The sub-tree of the call $\texttt{RS}(1, 4)$, which returns 32 to $\texttt{RS}(1, 7)$:



# Example (3/4)

The sub-tree of the call $\texttt{RS}(5, 7)$, which returns 25 to $\texttt{RS}(1, 7)$:



# Example (4/4)

- The root $\texttt{RS}(1, 7)$ computes $m = \lfloor 8/2 \rfloor = 4$ and calls $\texttt{RS}(1, 4)$ and $\texttt{RS}(5, 7)$
- $\texttt{RS}(1, 4)$ computes $m = 2$ and returns $17 + 15 = 32$; $\texttt{RS}(5, 7)$ computes $m = 6$ and returns $19 + 6 = 25$
- The root returns $32 + 25 = 57$, which is the sum of the elements of the array
- There are $13 = 2 \cdot 7 - 1$ calls: 7 leaves (base cases, one for each element) and 6 internal nodes, each of which performs one addition, so there are $6 = n - 1$ additions in total

# Depth-first order

- The calls are executed in **depth-first** order: a call is completed only after both its children are completed
- In the example the calls terminate in the order

$$(1,1),\ (2,2),\ (1,2),\ (3,3),\ (4,4),\ (3,4),\ (1,4),$$
$$(5,5),\ (6,6),\ (5,6),\ (7,7),\ (5,7),\ (1,7).$$

# Correctness

- The correctness of a recursive algorithm is proved by **strong induction** on the size of the sub-problem
- We assume that the recursive calls, which work on smaller sub-arrays, are correct, and we show that the call combines their results correctly

**Observation 1: correctness of RecursiveSum**

*For every pair of indices $l \leq r$, the call* `RecursiveSum`$(a, l, r)$ *terminates and returns*

$$\sum_{i=l}^{r} a[i].$$

# Proof (1/2)

We use strong induction on the number $k = r - l + 1 \geq 1$ of elements of the sub-array.

**Base case** ($k = 1$). Then $l = r$, the condition of line 2 is true and the algorithm returns

$$a[l] = \sum_{i=l}^{l} a[i].$$

**Inductive step** ($k \geq 2$). Assume that the statement holds for all sub-arrays with fewer than $k$ elements (*inductive hypothesis*). Since $l < r$, the algorithm computes $m = \lfloor (l + r)/2 \rfloor$, with $l \leq m < r$.

# Proof (2/2)

The sub-arrays $a[l..m]$ and $a[m+1..r]$ have $\lceil k/2 \rceil$ and $\lfloor k/2 \rfloor$ elements, both between 1 and $k-1$. By the inductive hypothesis, the two recursive calls terminate and

$$s_1 = \sum_{i=l}^{m} a[i], \qquad s_2 = \sum_{i=m+1}^{r} a[i].$$

Therefore, the algorithm returns

$$s_1 + s_2 = \sum_{i=l}^{m} a[i] + \sum_{i=m+1}^{r} a[i] = \sum_{i=l}^{r} a[i].$$

□

- With $l = 1$ and $r = n$, the call `RecursiveSum`$(a, 1, n)$ returns the sum of all the elements of the array

# Number of steps of `RecursiveSum`

- Let $S(n)$ denote the number of steps executed by `RecursiveSum` on a sub-array of $n = r - l + 1$ elements. As for the iterative algorithm, it depends only on $n$ (and not on the values of the elements)
- We count the lines of code executed **by a single call**
- A line containing a recursive call counts as 1 step (the call itself) **plus** all the steps executed by the called function, which we express again using $S$
- If $n = 1$, lines 2 and 3 are executed 1 time each, and therefore $S(1) = 2$

# Number of steps for $n \geq 2$

If $n \geq 2$:

- Line 2 is executed 1 time
- Line 3 is not executed
- Line 5 is executed 1 time
- Line 6 is executed 1 time, plus the $S(\lceil n/2 \rceil)$ steps of the call on $a[l..m]$
- Line 7 is executed 1 time, plus the $S(\lfloor n/2 \rfloor)$ steps of the call on $a[m+1..r]$
- Line 8 is executed 1 time

and therefore

$$S(n) = S(\lceil n/2 \rceil) + S(\lfloor n/2 \rfloor) + 5.$$

# The recurrence of `RecursiveSum`

The number of steps of `RecursiveSum` is described by the recurrence

$$S(n) = \begin{cases} 2 & \text{if } n = 1, \\ S(\lceil n/2 \rceil) + S(\lfloor n/2 \rfloor) + c & \text{if } n \geq 2, \end{cases}$$

with $c = 5$.

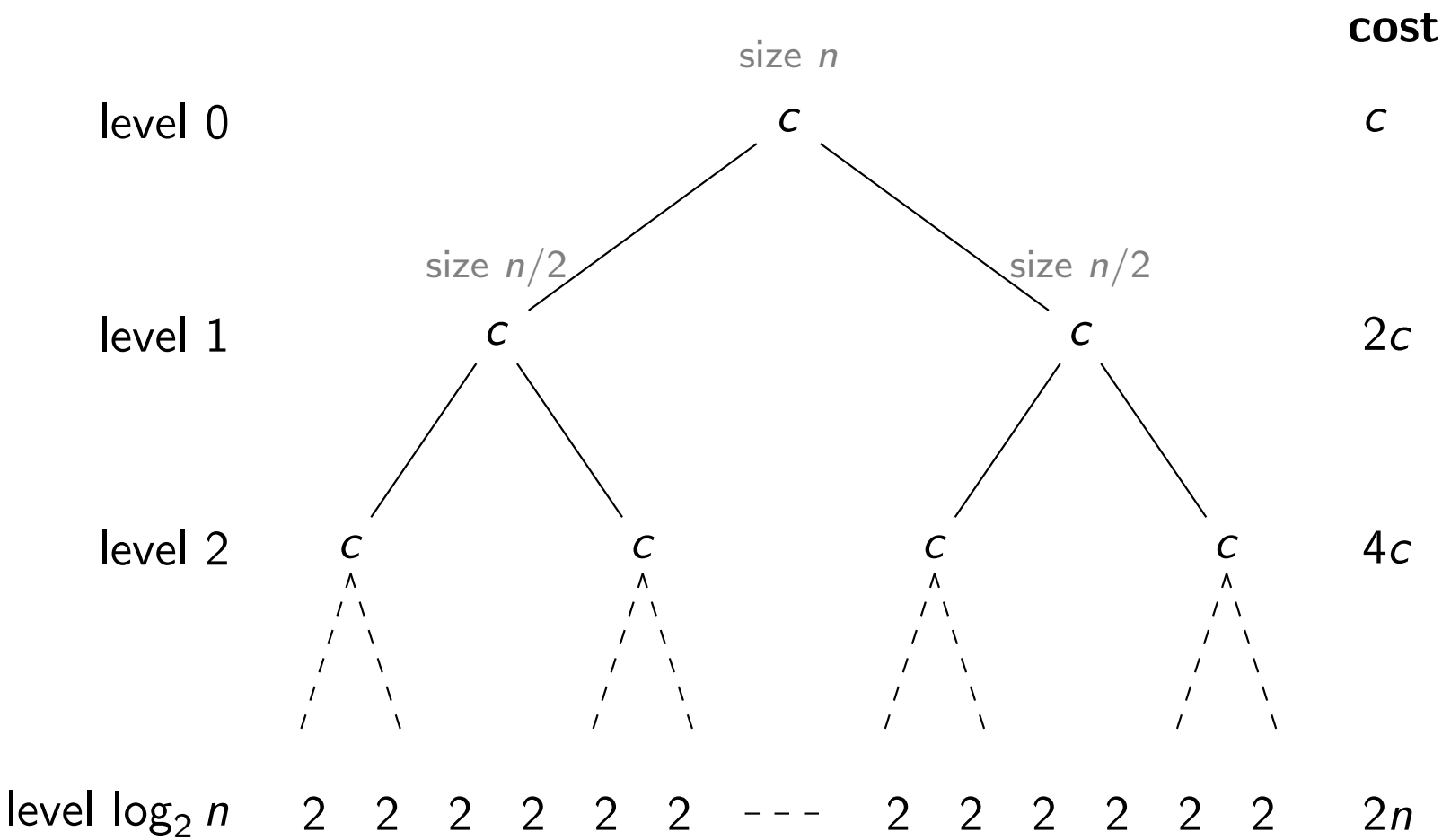
As usual, in the asymptotic analysis we write it simply as

$$S(n) = 2\,S(n/2) + c.$$

# Solution with the recursion tree

- In the **recursion tree** every node represents a call and contains the number of steps executed by that call *excluding* the steps of its recursive calls:
  - $c$ for an internal node
  - 2 for a leaf
- $S(n)$ is the sum of the values of all the nodes
- For simplicity, assume first that $n = 2^h$ is an exact power of 2. Then every sub-array is split into two halves of equal size, and the tree is shown in the next slide

# The recursion tree of $S(n) = 2\,S(n/2) + c$



# Counting the nodes

- Level $i$ contains $2^i$ calls on sub-arrays of $n/2^i$ elements
- The leaves are at level $\log_2 n$, where the sub-arrays have $n/2^{\log_2 n} = 1$ element: there are $2^{\log_2 n} = n$ leaves
- The internal levels $0, 1, \ldots, \log_2 n - 1$ contain

$$\sum_{i=0}^{\log_2 n - 1} 2^i = 2^{\log_2 n} - 1 = n - 1$$

  nodes (sum of a geometric series)
- Therefore the total number of steps is

$$S(n) = c\,(n-1) + 2n = 5(n-1) + 2n = 7n - 5.$$

# The leaves dominate

- The cost of the levels **doubles** from one level to the next one: the total is dominated by the leaves, which are $n$
- This is exactly the situation of **case 1** of the Master Theorem
- The same counting holds for **every** $n$, not only for powers of 2: the key fact is that the tree has exactly $n$ leaves and $n-1$ internal nodes, as the next observation shows

**Observation 2: calls and steps of RecursiveSum**

*For every $n \geq 1$, the call `RecursiveSum`$(a, l, r)$ on a sub-array of $n = r - l + 1$ elements generates exactly $2n-1$ calls (including the first one): $n$ of them are base cases and $n-1$ of them perform one addition. Moreover, $S(n) = 7n - 5$.*

# Proof (1/2)

Let $C(n)$ be the number of calls. We have $C(1) = 1$ and, for $n \geq 2$,

$$C(n) = 1 + C(\lceil n/2 \rceil) + C(\lfloor n/2 \rfloor).$$

We prove $C(n) = 2n - 1$ by strong induction on $n$.

For $n = 1$: $C(1) = 1 = 2 \cdot 1 - 1$.

For $n \geq 2$, by the inductive hypothesis applied to $\lceil n/2 \rceil < n$ and $\lfloor n/2 \rfloor < n$,

$$\begin{aligned} C(n) &= 1 + \big(2\lceil n/2 \rceil - 1\big) + \big(2\lfloor n/2 \rfloor - 1\big) \\ &= 2\big(\lceil n/2 \rceil + \lfloor n/2 \rfloor\big) - 1 = 2n - 1, \end{aligned}$$

because $\lceil n/2 \rceil + \lfloor n/2 \rfloor = n$ for every natural number $n$.

# Proof (2/2)

Each base case corresponds to exactly one element of the sub-array (the sub-arrays of the leaves are disjoint and cover $a[l..r]$), so there are $n$ base cases and

$$2n - 1 - n = n - 1$$

calls that execute line 8.

In the same way, $S(1) = 2 = 7 \cdot 1 - 5$ and, for $n \geq 2$,

$$S(n) = \left(7\lceil n/2 \rceil - 5\right) + \left(7\lfloor n/2 \rfloor - 5\right) + 5 = 7n - 5.$$

□

# $S(n)$ is linear

- Accordingly,

$$S(n) = 7n - 5 = \Theta(n), \qquad \text{since} \qquad \lim_{n \to \infty} \frac{7n-5}{n} = 7 \in \mathbb{R}_{>0}.$$

- For the array of the example, $S(7) = 44$:
  - 7 leaves with 2 steps each
  - 6 internal nodes with 5 steps each

# Solution with the Master Theorem

- We can obtain the order of growth of $S(n)$ directly from the Master Theorem, without computing $S(n)$ exactly
- We recall **case 1** of the Master Theorem:

Let $S(n) = a\,S(n/b) + f(n)$, with $a \geq 1$ and $b > 1$ constants and $f(n)$ asymptotically positive.

If $f(n) = O\left(n^{\log_b a - \varepsilon}\right)$ for some constant $\varepsilon > 0$, then

$$S(n) = \Theta\left(n^{\log_b a}\right).$$

# Case 1 for `RecursiveSum`

- $S(n) = 2\,S(n/2) + c \quad \rightarrow \quad a = 2,\ b = 2,\ f(n) = c$
- $n^{\log_b a} = n^{\log_2 2} = n^1 = n$
- Comparison: $f(n) = c$ grows polynomially slower than $n$ $\rightarrow$ case 1
- Hypothesis: $f(n) = c = O\left(n^{1-\varepsilon}\right)$ with $\varepsilon = 1$, since $c = c \cdot n^0$
- Conclusion: $S(n) = \Theta\left(n^{\log_2 2}\right) = \Theta(n)$

As expected, the result agrees with the exact solution $S(n) = 7n - 5$.

# Worst-case running time

- As in the chapter «Running time», we make the hypothesis that every entry of the array is stored using a fixed constant number of bits
- Then $|I| = \Theta(n)$, i.e., there exist constants $c_1, c_2 > 0$ such that $c_1 n \leq |I| \leq c_2 n$ for all sufficiently large $n$, and

$$\frac{1}{c_2}|I| \leq n \leq \frac{1}{c_1}|I|.$$

- Since $S(n) = 7n - 5 = \Theta(n)$, there exist constants $a_1, a_2 > 0$ such that $a_1 n \leq S(n) \leq a_2 n$ for all sufficiently large $n$, and hence

$$\frac{a_1}{c_2}|I| \leq S(n) \leq \frac{a_2}{c_1}|I|.$$

# Running time of the two algorithms

If all input values use a fixed constant number of bits, then the worst-case running time of the `RecursiveSum` algorithm (and also of the `SumIterative` algorithm) is

$$T(|I|) = \Theta(|I|).$$

Thus, both algorithms are linear in the input size.

- Recursion does not make the algorithm faster: both algorithms read every element once and perform $n-1$ (recursive) or $n$ (iterative) additions
- The recursive version even executes more steps ($7n-5$ against $3n+4$), because of the overhead of the calls

# Extra memory: the call stack

- Time is not the only resource
- Each active call of a recursive function needs its own **activation record** on the **call stack**, containing its parameters, its local variables and the return address
- In `RecursiveSum` the parameters and local variables are $l$, $r$, $m$, $s_1$, $s_2$: every activation record has constant size
- A call is active from the moment it starts until it returns. Since the calls are executed depth-first, the active calls at any moment form a **path** from the root of the call tree to the current node

# Recursion depth

The maximum number of simultaneously active calls
(the **recursion depth**)
equals the height of the call tree plus one.

The extra memory used by a recursive algorithm
is (at least) proportional to the recursion depth.

## Observation 3: recursion depth of RecursiveSum

*For every $n \geq 1$, the call tree of* `RecursiveSum` *on a sub-array of $n$ elements has height $\lceil \log_2 n \rceil$. Hence the recursion depth is $\lceil \log_2 n \rceil + 1$ and the extra memory used by* `RecursiveSum` *is $\Theta(\log_2 n)$.*

- The proof is again by strong induction, on the height $h(n)$ of the call tree

# Proof (1/2)

Let $h(n)$ be the height of the call tree. Then $h(1) = 0$ and, for $n \geq 2$,

$$h(n) = 1 + \max\left\{h(\lceil n/2 \rceil),\, h(\lfloor n/2 \rfloor)\right\}.$$

We prove $h(n) = \lceil \log_2 n \rceil$ by strong induction.

For $n = 1$, $\lceil \log_2 1 \rceil = 0$.

Let $n \geq 2$. Since $\lceil \log_2 x \rceil$ is non-decreasing in $x$, by the inductive hypothesis the maximum is attained by the larger half, and

$$h(n) = 1 + \left\lceil \log_2 \lceil n/2 \rceil \right\rceil.$$

# Proof (2/2)

Let $k = \lceil \log_2 n \rceil \geq 1$, i.e.,

$$2^{k-1} < n \leq 2^k.$$

Dividing by 2, we obtain $2^{k-2} < n/2 \leq 2^{k-1}$; since $2^{k-1}$ is an integer, also

$$2^{k-2} < \lceil n/2 \rceil \leq 2^{k-1},$$

that is $\lceil \log_2 \lceil n/2 \rceil \rceil = k - 1$. Therefore

$$h(n) = 1 + (k-1) = k = \lceil \log_2 n \rceil.$$

Each activation record has constant size, so the extra memory is $\Theta(\lceil \log_2 n \rceil + 1) = \Theta(\log_2 n)$. □

# Time and memory of the two algorithms

- In the example, $n = 7$ and the recursion depth is $\lceil \log_2 7 \rceil + 1 = 3 + 1 = 4$
- For instance, when $\texttt{RS}(1, 1)$ is executed, the active calls are $\texttt{RS}(1, 7)$, $\texttt{RS}(1, 4)$, $\texttt{RS}(1, 2)$ and $\texttt{RS}(1, 1)$

| algorithm | number of steps | extra memory |
|---|---|---|
| `SumIterative` | $3n + 4 = \Theta(n)$ | $\Theta(1)$ |
| `RecursiveSum` | $7n - 5 = \Theta(n)$ | $\Theta(\log_2 n)$ |

# A different recursion: decrease by one

- The halving is not the only way to sum an array recursively
- The array can also be split in an **unbalanced** way: the sum of $a[1..n]$ is the sum of $a[1..n-1]$ plus $a[n]$
- This is a *decrease by one* recursion, whose pseudo-code is in the next slide

**Algorithm:** RecursiveSumLinear

**Input :** An array $a[1..n]$ of $n \geq 1$ integer numbers
**Output :** The sum $a[1] + a[2] + \cdots + a[n]$

```
1 function RecursiveSumLinear(a, n)
2     if n = 1 then
3         return a[1]                                  // Base case
4     end
5     s := RecursiveSumLinear(a, n − 1)
6     return s + a[n]
7 end
```

Run in Colab: Python · C++ · Java

# Correctness and recurrence

- Correctness follows by (simple) induction on $n$:
  - for $n = 1$ the algorithm returns $a[1]$
  - for $n \geq 2$, by the inductive hypothesis $s = a[1] + \cdots + a[n-1]$, and the algorithm returns $s + a[n] = a[1] + \cdots + a[n]$

- If $n = 1$, lines 2 and 3 are executed once: $S(1) = 2$

- If $n \geq 2$, lines 2, 5 and 6 are executed once, plus the $S(n-1)$ steps of the recursive call:

$$S(n) = \begin{cases} 2 & \text{if } n = 1, \\ S(n-1) + 3 & \text{if } n \geq 2. \end{cases}$$

# Solution by iteration

- The Master Theorem **cannot** be applied, because the size of the sub-problem is $n-1$ and not $n/b$ for a constant $b > 1$
- We solve the recurrence by iteration (unrolling):

$$\begin{aligned} S(n) &= S(n-1) + 3 = S(n-2) + 2 \cdot 3 = \cdots \\ &= S(1) + (n-1) \cdot 3 = 3n - 1 = \Theta(n). \end{aligned}$$

- The call tree is a **path** of $n$ nodes: `RecursiveSumLinear`$(a, n)$ calls `RecursiveSumLinear`$(a, n-1)$, which calls `RecursiveSumLinear`$(a, n-2)$, and so on, until $n = 1$
- When the base case is reached, all the $n$ calls are active

# The price of the decrease-by-one recursion

- `RecursiveSumLinear` executes $S(n) = 3n - 1 = \Theta(n)$ steps, but its recursion depth is $n$
- It needs $\Theta(n)$ extra memory for the call stack, against $\Theta(\log_2 n)$ of `RecursiveSum` and $\Theta(1)$ of `SumIterative`
- In practice, for large $n$ (e.g. $n = 10^7$) the decrease-by-one version may exhaust the call stack (*stack overflow*), while `RecursiveSum` needs only

$$\lceil \log_2 10^7 \rceil + 1 = 25$$

active calls

# Summary

**Key points on the sum of an array**:

- `RecursiveSum` is correct (strong induction), makes $2n-1$ calls and executes $7n-5 = \Theta(n)$ steps: recursion does not reduce the work, which is linear because every element must be read

**Recurrence**:

- $S(n) = 2S(n/2) + c$ falls in case 1 of the Master Theorem: the cost is dominated by the $n$ leaves

**Memory**:

- The call stack costs memory: $\Theta(\log_2 n)$ for the halving recursion, $\Theta(n)$ for the decrease-by-one recursion, $\Theta(1)$ for the loop

# Merge Sort

# Introduction: Merge Sort

- For the sum of an array, divide and conquer gave no gain in time. We now meet a problem where it does make a difference: **sorting**
- Merge Sort splits the array in two halves, sorts them recursively and **merges** the two sorted halves
- Its number of steps is $\Theta(n \log_2 n)$ in the worst case, against $\Theta(n^2)$ of Insertion Sort: the price is $\Theta(n)$ extra memory

# The sorting problem

Given an array $a[1..n]$ of $n$ natural numbers,
we consider the problem of rearranging the elements of the array
in non-decreasing order such that:

$$a[1] \leq a[2] \leq \cdots \leq a[n].$$

- The problem has been introduced in the chapter «Sorting algorithms» (Insertion Sort, Exchange Sort and Selection Sort)
- In the chapter «Running time» we proved that the worst-case number of steps of `InsertionSort` is $S(n) = \frac{3n^2+9n-8}{2} = \Theta(n^2)$
- **Merge Sort**: a recursive comparison-based sorting algorithm based on **divide and conquer**, with $\Theta(n \log_2 n)$ steps in the worst case

# Example

We consider the following array of $n = 7$ natural numbers:

| $a[i]$ | $i = 1$ | $i = 2$ | $i = 3$ | $i = 4$ | $i = 5$ | $i = 6$ | $i = 7$ |
|---|---|---|---|---|---|---|---|
| | 9 | 12 | 3 | 16 | 19 | 11 | 12 |

The sorted array is:

| $a[i]$ | $i = 1$ | $i = 2$ | $i = 3$ | $i = 4$ | $i = 5$ | $i = 6$ | $i = 7$ |
|---|---|---|---|---|---|---|---|
| | 3 | 9 | 11 | 12 | 12 | 16 | 19 |

# Divide and conquer

- Merge Sort sorts a sub-array $a[l..r]$ as follows:
  1. **Divide**: split $a[l..r]$ in two halves $a[l..m]$ and $a[m+1..r]$, where $m = \lfloor (l+r)/2 \rfloor$;
  2. **Conquer**: sort recursively the two halves;
  3. **Combine**: **merge** the two sorted halves into a single sorted sub-array $a[l..r]$.

- The **base case** is a sub-array with a single element ($l = r$), which is already sorted: nothing has to be done

- The whole work is done in the **combine** step, by the procedure `Merge`

- Key observation: two *sorted* sequences can be merged very efficiently. The smallest element of the result is the smaller of the two first elements, and so on

This is the pseudo-code of the `MergeSort` algorithm.
To sort the whole array $a[1..n]$, we call `MergeSort`$(a, 1, n)$.

**Algorithm:** MergeSort

**Input :** An array $a$ and two indices $l \leq r$
**Output :** The sub-array $a[l..r]$ sorted in non-decreasing order

```
1 function MergeSort(a, l, r)
2     if l < r then
3         m := ⌊(l + r)/2⌋                          // Divide
4         MergeSort(a, l, m)          // Conquer (left half)
5         MergeSort(a, m + 1, r)      // Conquer (right half)
6         Merge(a, l, m, r)                         // Combine
7     end
8 end
```

Run in Colab: Python · C++ · Java

# The procedure Merge

- If $k = r - l + 1 \geq 2$, the two halves $a[l..m]$ and $a[m+1..r]$ have $\lceil k/2 \rceil$ and $\lfloor k/2 \rfloor$ elements, both between 1 and $k-1$: the recursion always reaches the base case
- The procedure Merge$(a, l, m, r)$ assumes that $a[l..m]$ and $a[m+1..r]$ are sorted
- It copies them into two auxiliary arrays $L$ and $R$, and then it repeatedly moves the smaller of the first two elements not yet copied back into $a$
- Its pseudo-code is long: we split it over four slides, with the numbering of the lines that continues

**Algorithm:** Merge (1/4)

**Input** : An array $a$ and indices $l \leq m < r$ such that $a[l..m]$ and $a[m+1..r]$ are sorted

**Output** : The sub-array $a[l..r]$ sorted in non-decreasing order

```
1  function Merge(a, l, m, r)
2      n_1 := m − l + 1
3      n_2 := r − m
4      L := new array[1..n_1], R := new array[1..n_2]
5      for i := 1 to n_1 do
6          L[i] := a[l + i − 1]
7      end
8      for j := 1 to n_2 do
9          R[j] := a[m + j]
10     end
11     i := 1
12     j := 1
```

## Algorithm: Merge (2/4)

**function** *Merge(a, l, m, r)* (continued)

```
13    k := l
14    while i ≤ n1 and j ≤ n2 do
15        if L[i] ≤ R[j] then
16            a[k] ← L[i]
17            i ← i + 1
18        else
19            a[k] ← R[j]
20            j ← j + 1
21        end
22        k ← k + 1
23    end
```

**Algorithm:** Merge (3/4)

**function** *Merge(a, l, m, r)* (continued)

```
24  |   while i ≤ n_1 do                    // Copy the rest of L
25  |   |   a[k] ← L[i]
26  |   |   i ← i + 1
27  |   |   k ← k + 1
28  |   end
```

**Algorithm:** Merge (4/4)

**function** *Merge(a, l, m, r)* (continued)

```
29      while j ≤ n_2 do                    // Copy the rest of R
30          a[k] ← R[j]
31          j ← j + 1
32          k ← k + 1
33      end
34  end
```

Run in Colab: Python · C++ · Java

# How Merge works (1/2)

- Lines 2–9 copy $a[l..m]$ into $L[1..n_1]$ and $a[m+1..r]$ into $R[1..n_2]$
- The copies are necessary, because the positions $a[l..r]$ are overwritten during the merge
- The indices $i$ and $j$ point to the first elements of $L$ and $R$ not yet copied back, and $k$ points to the next position of $a$ to be filled

# How Merge works (2/2)

- The first while loop (line 14) compares $L[i]$ and $R[j]$ and copies the smaller one into $a[k]$
- In case of a tie, the element of $L$ is copied first: this makes Merge Sort **stable**
- As soon as one of the two arrays is exhausted, the remaining elements of the other one (which are already sorted, and not smaller than the ones already copied) are copied into $a$ by the while loop of line 24 or by the one of line 29
- Only one of these two loops executes at least one iteration

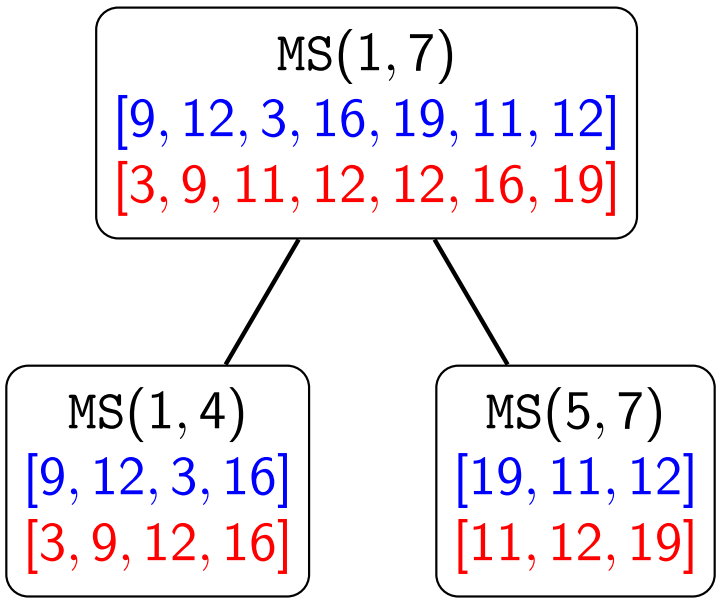
# Examples

We now run Merge Sort on the array $a = [9, 12, 3, 16, 19, 11, 12]$:

first we look at the whole call tree,
then at the two phases of the algorithm,
and finally at a single call of `Merge`.

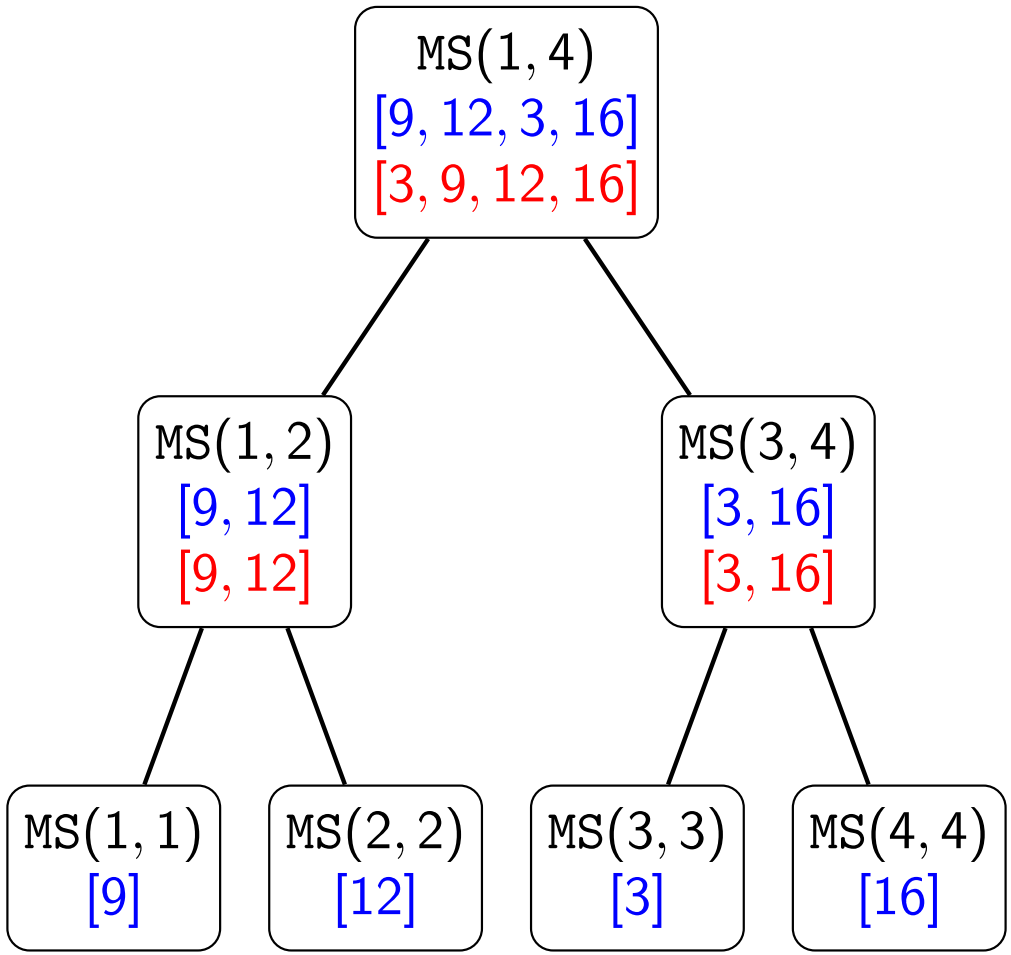
# Example: call tree (1/4)

- The call tree of `MergeSort`$(a, 1, 7)$: each node `MS`$(l, r)$ is a call on the sub-array $a[l..r]$
- In each node, in blue the sub-array received by the call and in red the sorted sub-array produced by the call (after its `Merge`)
- First the root and its two children, then the sub-trees of `MS`$(1, 4)$ and of `MS`$(5, 7)$



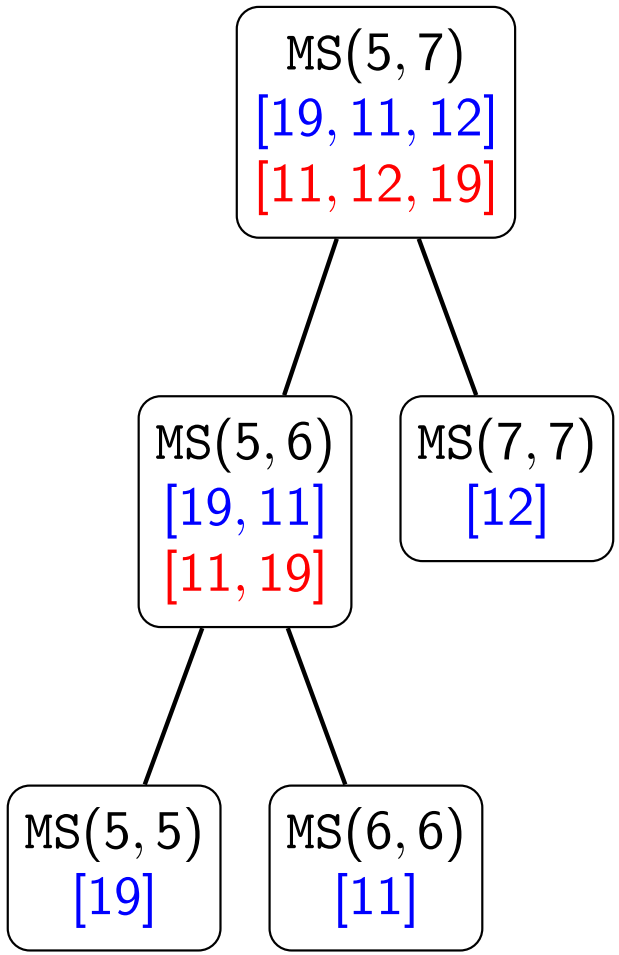
# Example: call tree (2/4)

The sub-tree of the call $\texttt{MS}(1, 4)$:



# Example: call tree (3/4)

The sub-tree of the call $\texttt{MS}(5, 7)$:



# Example: call tree (4/4)

The calls of `Merge` are executed in the following order (depth-first), and this is the content of the array after each of them (the sub-array modified by each call is in red):

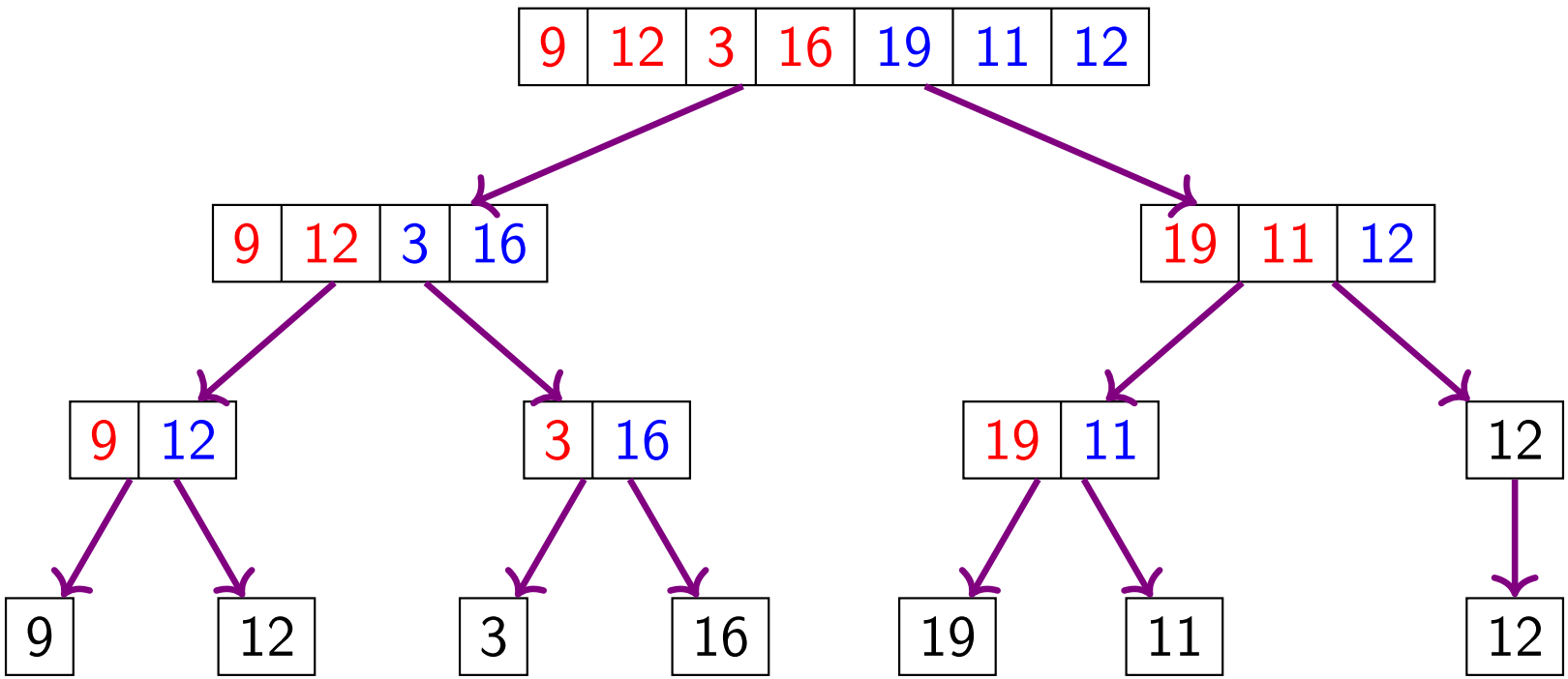
| call | merged sub-arrays | array $a$ after the call |
|---|---|---|
| `Merge`$(a, 1, 1, 2)$ | $[9]$ and $[12]$ | $[9, 12, 3, 16, 19, 11, 12]$ |
| `Merge`$(a, 3, 3, 4)$ | $[3]$ and $[16]$ | $[9, 12, 3, 16, 19, 11, 12]$ |
| `Merge`$(a, 1, 2, 4)$ | $[9, 12]$ and $[3, 16]$ | $[3, 9, 12, 16, 19, 11, 12]$ |
| `Merge`$(a, 5, 5, 6)$ | $[19]$ and $[11]$ | $[3, 9, 12, 16, 11, 19, 12]$ |
| `Merge`$(a, 5, 6, 7)$ | $[11, 19]$ and $[12]$ | $[3, 9, 12, 16, 11, 12, 19]$ |
| `Merge`$(a, 1, 4, 7)$ | $[3, 9, 12, 16]$ and $[11, 12, 19]$ | $[3, 9, 11, 12, 12, 16, 19]$ |

# Example: divide and combine phases (1/3)

- We consider again the array $a = [9, 12, 3, 16, 19, 11, 12]$
- The **divide** phase (violet arrows): each sub-array is split into its left half (in red) and its right half (in blue), until sub-arrays of one element are obtained
- The **combine** phase (orange arrows): pairs of sorted sub-arrays are merged into a sorted sub-array

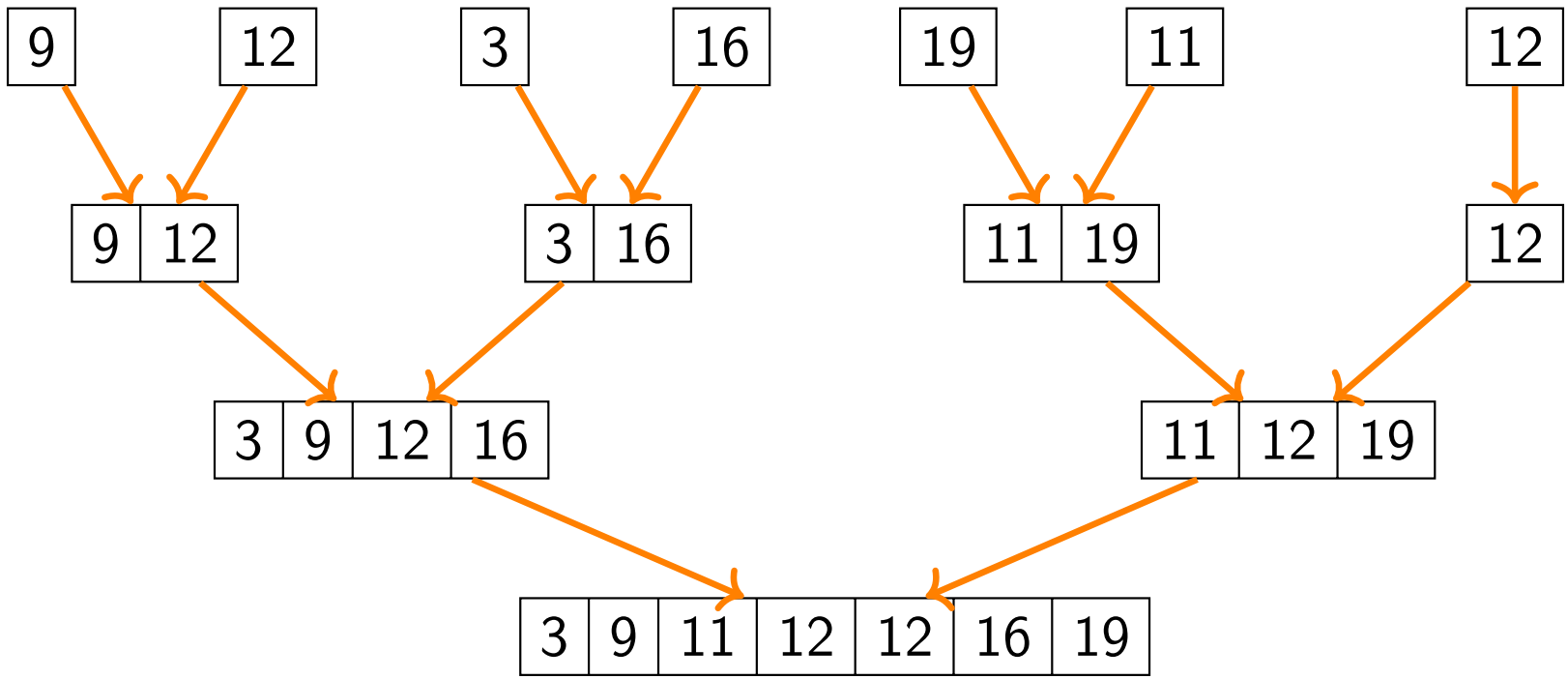
# Example: divide and combine phases (2/3)

The **divide** phase:



# Example: divide and combine phases (3/3)

The **combine** phase:



# Example: trace of `Merge` (1/2)

- We consider the last call `Merge`$(a, 1, 4, 7)$ of the call tree
- Before the call $a = [3, 9, 12, 16, 11, 12, 19]$, with $a[1..4]$ and $a[5..7]$ sorted
- Then $n_1 = 4$, $n_2 = 3$ and

$$L = [3, 9, 12, 16], \qquad R = [11, 12, 19].$$

- In the next slide, the execution of the while loops (one row per element copied into $a$)

# Example: trace of `Merge` (2/2)

| $k$ | $i$ | $j$ | $L[i]$ | $R[j]$ | comparison | action |
|---|---|---|---|---|---|---|
| 1 | 1 | 1 | 3 | 11 | $3 \leq 11$ | $a[1] \leftarrow 3$, $i = 2$ |
| 2 | 2 | 1 | 9 | 11 | $9 \leq 11$ | $a[2] \leftarrow 9$, $i = 3$ |
| 3 | 3 | 1 | 12 | 11 | $12 > 11$ | $a[3] \leftarrow 11$, $j = 2$ |
| 4 | 3 | 2 | 12 | 12 | $12 \leq 12$ | $a[4] \leftarrow 12$ (from $L$), $i = 4$ |
| 5 | 4 | 2 | 16 | 12 | $16 > 12$ | $a[5] \leftarrow 12$ (from $R$), $j = 3$ |
| 6 | 4 | 3 | 16 | 19 | $16 \leq 19$ | $a[6] \leftarrow 16$, $i = 5$ |
| 7 | 5 | 3 | – | 19 | $L$ exhausted | $a[7] \leftarrow 19$, $j = 4$ |

- After 6 iterations of the first while loop (and 6 comparisons), $i = 5 > n_1$: the array $L$ is exhausted, and the last element of $R$ is copied by the while loop of line 29
- The result is $a = [3, 9, 11, 12, 12, 16, 19]$

# Correctness

- The correctness of Merge Sort rests on two facts:
  - `Merge` correctly merges two sorted sub-arrays: a property of a loop, proved with a **loop invariant**
  - the recursion is correct: proved by **strong induction**, as for `RecursiveSum`

- A loop invariant is a property that holds at the beginning of every iteration of a loop. As in the chapter «Correctness», we show that the invariant
  - holds before the first iteration (*initialization*)
  - if it holds before an iteration, it also holds before the next one (*maintenance*)

  and what it gives us when the loop ends (*termination*)

## Observation 4: loop invariant of Merge

*Let $a[l..m]$ and $a[m+1..r]$ be sorted. At the beginning of every iteration of the while loop of line 14 of* `Merge`*:*

(I1) *the sub-array $a[l..k-1]$ contains the $k-l$ elements of $L[1..i-1]$ and $R[1..j-1]$, in non-decreasing order;*

(I2) *every element of $a[l..k-1]$ is smaller than or equal to every element of $L[i..n_1]$ and of $R[j..n_2]$.*

*At the end,* `Merge` *terminates and $a[l..r]$ contains the elements originally in $a[l..r]$, sorted in non-decreasing order.*

# Proof (1/4)

After lines 2–9, $L[1..n_1]$ and $R[1..n_2]$ are copies of the sorted sub-arrays $a[l..m]$ and $a[m+1..r]$, hence they are sorted.

**Initialization.** Before the first iteration $k = l$, $i = 1$, $j = 1$: the sub-array $a[l..k-1]$ is empty, and (I1) and (I2) hold trivially.

# Proof (2/4)

**Maintenance.** Assume (I1) and (I2) at the beginning of an iteration, and suppose $L[i] \leq R[j]$ (the other case is symmetric).

- Since $L$ and $R$ are sorted,

  $$L[i] \leq L[i+1..n_1] \qquad \text{and} \qquad L[i] \leq R[j] \leq R[j+1..n_2] :$$

  $L[i]$ is the smallest element not yet copied

- By (I2), $L[i]$ is greater than or equal to all the elements of $a[l..k-1]$, so after $a[k] \leftarrow L[i]$ the sub-array $a[l..k]$ is sorted and contains $L[1..i]$ and $R[1..j-1]$

# Proof (3/4)

- After $i \leftarrow i + 1$ and $k \leftarrow k + 1$, (I1) holds again
- (I2) holds as well, because the elements not yet copied are a subset of the previous ones, and the new element $a[k-1] = L[i-1]$ is smaller than or equal to all of them

# Proof (4/4)

**Termination.** Every iteration increases $i + j$ by one, so the loop ends after at most $n_1 + n_2$ iterations, with $i = n_1 + 1$ or $j = n_2 + 1$.

Suppose $i = n_1 + 1$ (the other case is symmetric).

- The loop of line 24 is skipped, and the loop of line 29 copies $R[j..n_2]$, in this order, into $a[k..r]$
- Since $R[j..n_2]$ is sorted and, by (I2), its elements are not smaller than those of $a[l..k-1]$, the whole sub-array $a[l..r]$ is sorted
- By (I1), it contains exactly the elements of $L$ and $R$, i.e., the elements originally in $a[l..r]$

□

# Correctness of `MergeSort`

Now that `Merge` is correct,
the correctness of the recursion follows by strong induction.

**Observation 5: correctness of MergeSort**

*For every pair of indices $l \leq r$, the call `MergeSort`$(a, l, r)$ terminates and rearranges the elements of $a[l..r]$ in non-decreasing order (without modifying the rest of the array).*

# Proof (1/2)

We use strong induction on the number $k = r - l + 1 \geq 1$ of elements of the sub-array.

**Base case** ($k = 1$). Then $l = r$, the condition of line 2 is false and nothing is done: a sub-array of one element is sorted.

**Inductive step** ($k \geq 2$). Assume that the statement holds for all sub-arrays with fewer than $k$ elements. Since $l < r$, the algorithm computes $m = \lfloor (l + r)/2 \rfloor$, with $l \leq m < r$.

# Proof (2/2)

- The sub-arrays $a[l..m]$ and $a[m+1..r]$ have $\lceil k/2 \rceil$ and $\lfloor k/2 \rfloor$ elements, both between 1 and $k-1$
- By the inductive hypothesis, after lines 4 and 5 both $a[l..m]$ and $a[m+1..r]$ are sorted (and contain the same elements as before)
- Therefore the hypothesis of `Merge` is satisfied, and by Observation 4, after line 6 the sub-array $a[l..r]$ is sorted

□

# Number of steps of `Merge`

- Let $n = r - l + 1 = n_1 + n_2$ be the number of elements to be merged
- Let $t$ be the number of iterations of the while loop of line 14. Among them, $t_L$ copy an element of $L$ and $t_R$ copy an element of $R$, with $t_L + t_R = t$
- The header of a for loop from 1 to $n_1$ is executed $n_1 + 1$ times (the last test fails)
- The line `else` is part of the `if` of line 15 and is not counted separately

# Number of executions of each line

The number of times each line of code is executed (each line of a group the same number of times):

| lines | executions |
|---|---|
| 2, 3, 4 | 1 |
| 5 | $n_1 + 1$ |
| 6 | $n_1$ |
| 8 | $n_2 + 1$ |
| 9 | $n_2$ |
| 11, 12, 13 | 1 |
| 14 | $t + 1$ |
| 15 | $t$ |

| lines | executions |
|---|---|
| 16, 17 | $t_L$ |
| 19, 20 | $t_R$ |
| 22 | $t$ |
| 24 | $(n_1 - t_L) + 1$ |
| 25, 26, 27 | $n_1 - t_L$ |
| 29 | $(n_2 - t_R) + 1$ |
| 30, 31, 32 | $n_2 - t_R$ |

# Total number of steps of `Merge`

Therefore, the total number of steps of `Merge` is

$$\begin{aligned} &3 + (2n_1 + 1) + (2n_2 + 1) + 3 + (t + 1) + t + 2t_L + 2t_R + t \\ &\quad + 4(n_1 - t_L) + 1 + 4(n_2 - t_R) + 1 \\ &= 11 + 2n + 5t + 4(n - t) = 6n + t + 11. \end{aligned}$$

- Every iteration of the first while loop copies one element into $a$, so $t \leq n$. More precisely:
  - the loop stops as soon as one of the two arrays is exhausted, so $t \geq \min\{n_1, n_2\}$
  - when the loop stops, the other array still contains at least one element, so $t \leq n - 1$

# Bounds on the number of steps of `Merge`

- Hence the number of steps of `Merge` on $n$ elements is between $6n + 11 + \min\{n_1, n_2\}$ and $7n + 10$; in particular, it is between $6n + 11$ and $7n + 10$
- The worst case $t = n - 1$ happens, for instance, in the trace of `Merge` seen above ($n = 7$, $t = 6$, $6 \cdot 7 + 6 + 11 = 59$ steps)

The number of steps of `Merge` on $n = r - l + 1$ elements is $6n + t + 11$ and satisfies

$$6n+11 \;\leq\; S_{\texttt{Merge}}(n) \;\leq\; 7n+10, \qquad \text{hence} \qquad S_{\texttt{Merge}}(n) = \Theta(n).$$

The number of comparisons between elements (line 15) is $t$, with $\min\{n_1, n_2\} \leq t \leq n - 1$.

# The recurrence of `MergeSort` (1/2)

- Let $S(n)$ denote the number of steps executed by `MergeSort` in the worst case on a sub-array of $n = r - l + 1$ elements
- As for the sum of an array, a line containing a call counts 1 step plus the steps of the called function
- If $n = 1$, only line 2 is executed: $S(1) = 1$
- If $n \geq 2$:
  - Line 2 is executed 1 time
  - Line 3 is executed 1 time
  - Line 4 is executed 1 time, plus the $S(\lceil n/2 \rceil)$ steps of the call
  - Line 5 is executed 1 time, plus the $S(\lfloor n/2 \rfloor)$ steps of the call
  - Line 6 is executed 1 time, plus the $S_{\texttt{Merge}}(n)$ steps of `Merge`

# The recurrence of `MergeSort` (2/2)

- Therefore

  $$S(n) = S(\lceil n/2 \rceil) + S(\lfloor n/2 \rfloor) + f(n), \qquad f(n) = 5 + S_{\texttt{Merge}}(n).$$

- From the analysis of `Merge`, $6n + 16 \leq f(n) \leq 7n + 15$
- Since $7n + 15 \leq 15n$ for $n \geq 2$, we have

  $$6n \leq f(n) \leq 15\,n \qquad \text{for every } n \geq 2,$$

  i.e., $f(n) = \Theta(n)$

# The recurrence of `MergeSort`

The number of steps of `MergeSort` is described by the recurrence

$$S(n) = \begin{cases} 1 & \text{if } n = 1, \\ S\big(\lceil n/2 \rceil\big) + S\big(\lfloor n/2 \rfloor\big) + \Theta(n) & \text{if } n \geq 2, \end{cases}$$

which, ignoring floors and ceilings, we write as

$$S(n) = 2\,S(n/2) + \Theta(n).$$

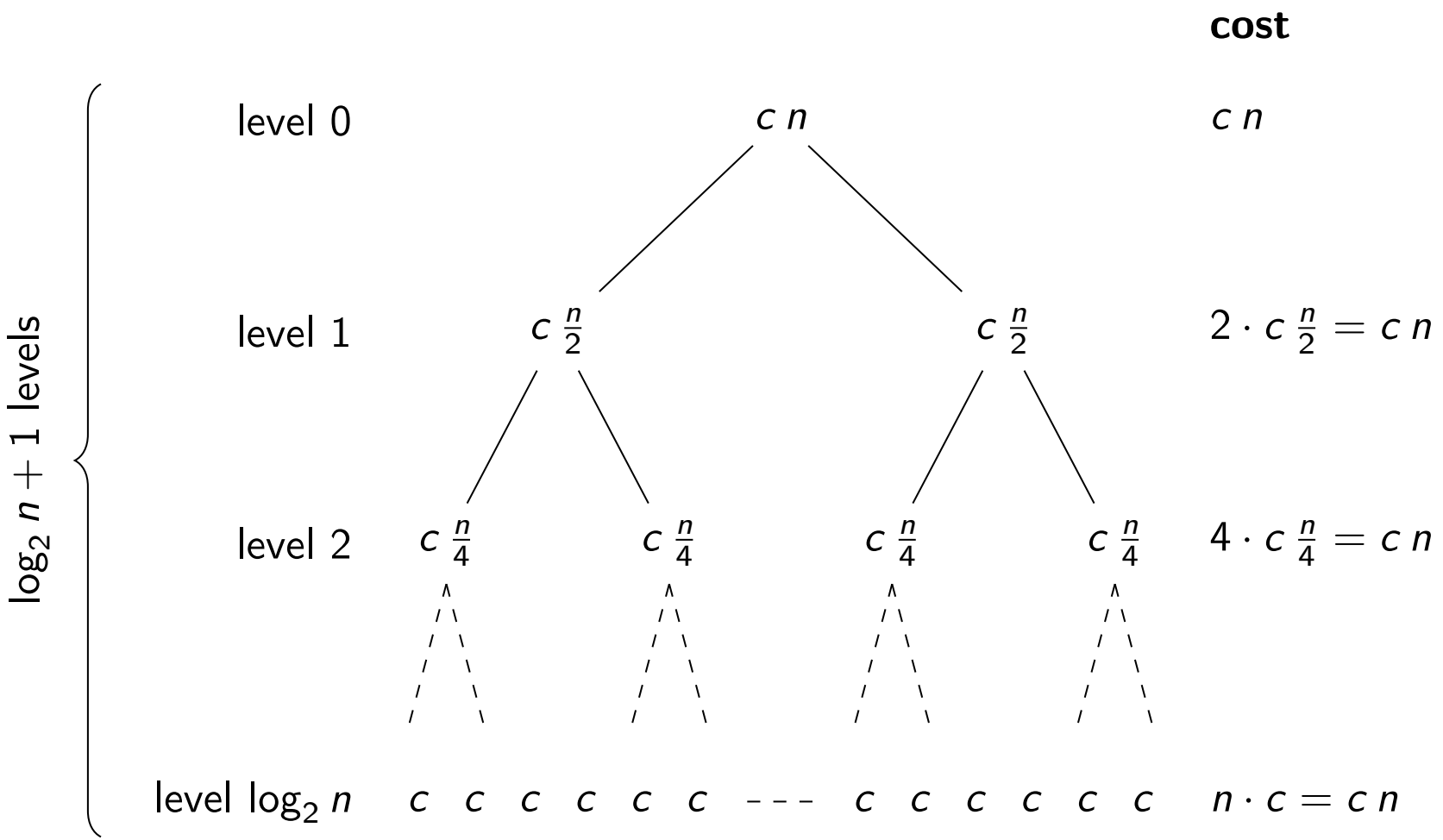
# Solution with the recursion tree

- Assume that $n = 2^h$ is an exact power of 2, and let $c = 15$ be the constant such that $f(n) \leq c\,n$ (with $S(1) = 1 \leq c$)
- We first bound $S(n)$ from above by the recursion tree of

$$S(n) \leq 2\,S(n/2) + c\,n,$$

  where each node contains the cost of a call, excluding the cost of its recursive calls
- The tree is in the next slide

# The recursion tree of $S(n) \leq 2\,S(n/2) + c\,n$



# The cost of the levels

- The top level has cost $c\,n$; the next level has cost $c\frac{n}{2} + c\frac{n}{2} = c\,n$; the level after has cost $4 \cdot c\frac{n}{4} = c\,n$, and so on
- In general, level $i$ has $2^i$ nodes, each corresponding to a sub-array of $n/2^i$ elements and contributing a cost of at most $c\,n/2^i$, so each level costs at most

$$2^i \cdot c\,\frac{n}{2^i} = c\,n.$$

- The bottom level $\log_2 n$ has $n$ leaves (sub-arrays of one element), each with cost $S(1) = 1 \leq c$: its cost is at most $c\,n$
- The levels are $0, 1, \ldots, \log_2 n$, i.e., $\log_2 n + 1$. Summing the costs of all the levels:

$$S(n) \leq c\,n\,(\log_2 n + 1) = c\,n \log_2 n + c\,n.$$

# $S(n) = \Theta(n \log_2 n)$

- In the same way, using $f(n) \geq 6n$, each internal level costs at least $6n$, and there are $\log_2 n$ internal levels: $S(n) \geq 6\, n \log_2 n$
- Therefore, for $n$ a power of 2,

$$6\, n \log_2 n \;\leq\; S(n) \;\leq\; 15\, n \log_2 n + 15\, n,$$

i.e., $S(n) = \Theta(n \log_2 n)$, since

$$\lim_{n\to\infty} \frac{15\, n \log_2 n + 15\, n}{n \log_2 n} = \lim_{n\to\infty} \left(15 + \frac{15}{\log_2 n}\right) = 15 \in \mathbb{R}_{>0}.$$

# All the levels have the same cost

- Contrary to the recursion tree of the sum of an array, where the cost of the levels doubles at each level, here **all the levels have the same cost** $c\,n$: the total is the cost of one level times the number of levels
- This is exactly the situation of **case 2** of the Master Theorem

# Solution with the Master Theorem

We recall **case 2** of the Master Theorem:

Let $S(n) = a\,S(n/b) + f(n)$, with $a \geq 1$ and $b > 1$ constants and $f(n)$ asymptotically positive.

If $f(n) = \Theta\left(n^{\log_b a} \log^k n\right)$ for some constant $k \geq 0$, then

$$S(n) = \Theta\left(n^{\log_b a} \log^{k+1} n\right).$$

# Case 2 for MergeSort

- $S(n) = 2\,S(n/2) + \Theta(n) \quad \rightarrow \quad a = 2,\ b = 2,\ f(n) = \Theta(n)$
- $n^{\log_b a} = n^{\log_2 2} = n$
- Comparison: $f(n)$ and $n^{\log_b a}$ grow in the same way $\rightarrow$ case 2
- Hypothesis: $f(n) = \Theta(n) = \Theta\left(n^{\log_2 2} \log^0 n\right)$ with $k = 0$
- Conclusion: $S(n) = \Theta\left(n^{\log_2 2} \log^1 n\right) = \Theta(n \log n)$

- The base of the logarithm is irrelevant inside $\Theta$, because $\log_b n = \frac{\log_2 n}{\log_2 b}$ differs from $\log_2 n$ by a constant factor
- Hence $S(n) = \Theta(n \log_2 n)$, in agreement with the recursion tree

# Worst-case running time (1/2)

- We make the hypothesis that every entry of the array is stored using a fixed constant number of bits
- Then $|I| = \Theta(n)$: there exist constants $c_1, c_2 > 0$ such that, for all sufficiently large $n$,

$$\frac{1}{c_2}|I| \leq n \leq \frac{1}{c_1}|I|.$$

- Taking logarithms,
  $\log_2 |I| - \log_2 c_2 \leq \log_2 n \leq \log_2 |I| - \log_2 c_1$, hence, for sufficiently large $|I|$,

$$\frac{1}{2}\log_2 |I| \leq \log_2 n \leq 2\log_2 |I|.$$

# Worst-case running time (2/2)

- Multiplying the two chains of inequalities (all terms are positive),

$$\frac{1}{2c_2}\,|I|\log_2|I| \;\leq\; n\log_2 n \;\leq\; \frac{2}{c_1}\,|I|\log_2|I|,$$

  that is, $n\log_2 n = \Theta(|I|\log_2|I|)$

- Since $S(n) = \Theta(n\log_2 n)$, we obtain $S(n) = \Theta(|I|\log_2|I|)$

If all input values use a fixed constant number of bits, then the worst-case running time of the `MergeSort` algorithm is

$$T(|I|) = \Theta(|I|\log|I|).$$

# Extra memory

- `Merge`$(a, l, m, r)$ allocates the auxiliary arrays $L$ and $R$, with $n_1 + n_2 = r - l + 1$ elements in total, which are released when `Merge` returns
- `Merge` is called only after both recursive calls have returned, so at any moment at most one `Merge` is active, and the largest auxiliary arrays are the ones of the first call `MergeSort`$(a, 1, n)$, with $n$ elements in total
- In addition, the call stack contains at most $\lceil \log_2 n \rceil + 1$ activation records of constant size (the call tree has the same shape as the one of `RecursiveSum`, Observation 3): $\Theta(\log_2 n)$ memory

# Merge Sort is not in place

`MergeSort` uses

$$\Theta(n) + \Theta(\log_2 n) = \Theta(n)$$

extra memory: it is **not** an in-place sorting algorithm.

On the contrary, `InsertionSort` sorts in place,
with $\Theta(1)$ extra memory.

# Comparison with Insertion Sort

- In the worst case, `InsertionSort` executes $\Theta(n^2)$ steps, while `MergeSort` executes $\Theta(n \log_2 n)$ steps. Since

$$\lim_{n \to \infty} \frac{n \log_2 n}{n^2} = \lim_{n \to \infty} \frac{\log_2 n}{n} = 0,$$

  Merge Sort is asymptotically much faster

- To get a feeling of the difference, we ignore the multiplicative constants and assume that a computer executes $10^9$ steps per second: see the next slide

# $n^2$ against $n \log_2 n$

| $n$ | $n^2$ | $n \log_2 n$ | time for $n^2$ steps | time for $n \log_2 n$ steps |
|---|---|---|---|---|
| 10 | 100 | $\approx 33$ | $10^{-7}$ s | $\approx 3.3 \cdot 10^{-8}$ s |
| $10^3$ | $10^6$ | $\approx 9\,966$ | $10^{-3}$ s | $\approx 10^{-5}$ s |
| $10^6$ | $10^{12}$ | $\approx 1.99 \cdot 10^7$ | $10^3$ s $\approx 16.7$ min | $\approx 0.02$ s |

- For small arrays the difference is negligible (and, because of the constants and of the cost of the recursive calls, `InsertionSort` can even be faster)
- For large arrays, Merge Sort is dramatically faster: with $n = 10^6$ elements it is about 50 000 times faster

# Summary

**Key points on Merge Sort**:

- All the work is done by `Merge`, which merges two sorted sub-arrays of total size $n$ in $6n + t + 11 = \Theta(n)$ steps
- $S(n) = 2S(n/2) + \Theta(n)$ falls in case 2 of the Master Theorem: $\log_2 n + 1$ levels of cost $\Theta(n)$ each, $S(n) = \Theta(n \log_2 n)$
- The price is memory: $\Theta(n)$ for the auxiliary arrays

| algorithm | worst-case number of steps | extra memory |
|---|---|---|
| `InsertionSort` | $\Theta(n^2)$ | $\Theta(1)$ |
| `MergeSort` | $\Theta(n \log_2 n)$ | $\Theta(n)$ |

# Summary of this lesson (1/3)

**The scheme of the analysis**:

- pseudo-code and call tree
- correctness by strong induction on the size of the sub-problem
- recurrence, solved with the recursion tree and with the Master Theorem
- running time as a function of $|I|$
- extra memory, including the call stack

# Summary of this lesson (2/3)

**The sum of an array**:

- `RecursiveSum`: $S(n) = 2S(n/2) + c$, $7n - 5 = \Theta(n)$ steps, $\Theta(\log_2 n)$ memory
- `RecursiveSumLinear`: $S(n) = S(n-1) + 3$, $3n - 1 = \Theta(n)$ steps, $\Theta(n)$ memory
- recursion gives no gain in time over `SumIterative`, and costs memory

# Summary of this lesson (3/3)

**Merge Sort**:

- divide in two halves, sort them recursively, **merge** them in $\Theta(n)$ steps
- correctness: a loop invariant for `Merge`, strong induction for the recursion
- $S(n) = 2S(n/2) + \Theta(n) = \Theta(n \log_2 n)$, against $\Theta(n^2)$ of `InsertionSort`
- $\Theta(n)$ extra memory: not in place

**Next lesson**:

- the Recursive Binary Search (one recursive call) and the maximum subarray problem